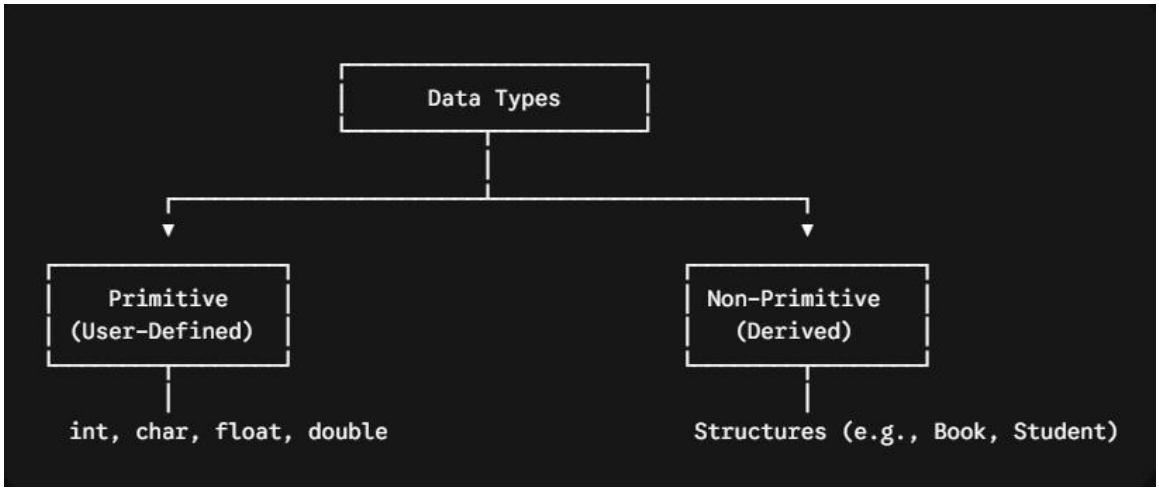
1. Introduction to Structures

What is a Structure?

A **Structure** is a user-defined (or secondary/non-primitive) data type in C that allows you to group variables of **dissimilar data types** under a single name.

Unlike an array, which holds elements of the same type, a structure acts as a container for heterogeneous data elements.

Custom Data Type Classification



2. Defining a Structure & Declaring Variables

Syntax and Templates

To define a structure, use the struct keyword followed by a tag name. Member variables declared inside the braces do not have separate initial identities or allocations until a structure variable is instantiated.

Template Definition

```
struct student
{
   char name[20];
   int rollno;
   float marks;
}; -------> Note the terminating semicolon

struct Date {
   int d, m, y;
};
```

Instantiating Structure Variables

A structure type can be declared globally or locally within functions.

Method 1: Declaring separately

struct Date d1, d2;

Method 2: Declaring immediately at definitionstruct

Date

{

int d, m, y;

} d1, d2;

Memory Allocation Layout

When a structure variable like struct Date d1; is declared, memory is assigned sequentially to its member elements. Assuming a standard 4-byte int:

3. Initialization and Member Access

Methods of Initialization

Sequential Initializer List: Elements must exactly follow the order of member declarations within the layout.

struct Date d1 = {28, 9, 2024};

⚠ **Note:** If the number of initializers is fewer than the total number of members, the remaining members are automatically initialized to **zero (0)**.

Designated / Dot Operator Access: Members are assigned explicitly using the member access operator (.) after declaration.

struct Date d1;
d1.d = 29;

```
d1.m = 9;
d1.y = 2024;
```

Reading Formatted Input via scanf

By default, scanf matches space, tab, or newline (\n) characters as delimiters. To enforce a specific input delimiter scheme (such as forward slashes /), explicitly declare them in the format string.

```
struct Date d1;
printf("Enter date (DD/MM/YYYY): ");  --> Capturing specific user delimiters safely
scanf("%d/%d/%d", &d1.d, &d1.m, &d1.y);
printf("Date: %d-%d-%d", d1.d, d1.m, d1.y);
```

4. Structures with Functions & Arrays

Data Operations: Assignment, Functions, and Arrays

Direct Copying: If two variables are instances of the exact same structure type, you can copy all properties smoothly via a direct assignment statement (d2 = d1;).

Function Passing: You can return structured data blocks from a function or supply them as complete parameter instances.

```
#include <stdio.h>
struct Date { int d, m, y; };
struct Date inputDate();   -----> Function Declarations
void showDate(struct Date newDate);
int main()
{
   struct Date d1 = inputDate();   ------> Function Return
   showDate(d1);  ------>  Pass by Value Instance
   return 0;
}
struct Date inputDate()
 {
```

```
    struct Date somedate;
    scanf("%d/%d/%d", &somedate.d, &somedate.m, &somedate.y);
    return somedate;
}
void showDate(struct Date newDate)
{
    printf("%d-%d-%d\n", newDate.d, newDate.m, newDate.y);
}
```

Arrays of Structures

An array of structures provisions contiguous blocks in memory where each individual slot represents an independent structural sequence.

struct Date dob[4]; **-----> Allocates a block for 4 Date objects**

Total Memory Calculation: 4 entries * 12 bytes/entry = 48 bytes total

5. Structure Pointers & Aliasing (typedef)

Referencing Structures via Pointers

When accessing structure items using an active object pointer, use the arrow operator (->) as a cleaner syntax alternative to bracketed indirection ((*ptr).member).

struct Date d1;struct Date *p = &d1;

The following statements are completely identical:

d1.d = 5; **-----> Direct Instance Access**
(*p).d = 5; **------> Pointer Dereference Access**
p->d = 5; **-------> Arrow Operator (Preferred notation)**

Aliasing via typedef

The typedef keyword creates a shorter alias for an existing long type definition, allowing you to emit the struct keyword during subsequent object declarations.

```
typedef struct student
{
    int rollno;
    char name[40];
    float marks;
```

```
} stu;  ------>  'stu' becomes the type alias name
int main()
{
   stu s = {10, "Gaurav", 80.94}; -->Clean declaration without the 'struct' keyword
   printf("%d %s %f", s.rollno, s.name, s.marks);
}
```

6. Memory Optimization: Padding vs. Packing

Structure Padding

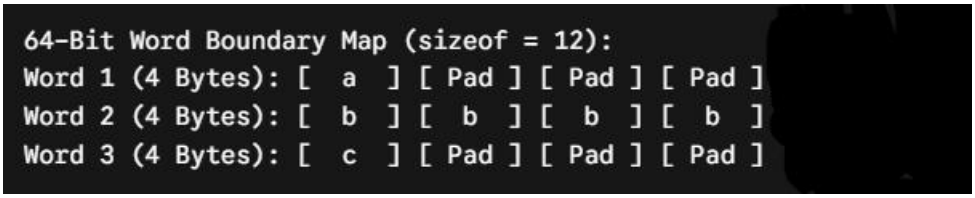
Processors do not read data from system memory 1 byte at a time; they read data in fixed alignments called **word sizes** (4 bytes at a time on 32-bit CPUs; 8 bytes at a time on 64-bit CPUs).

To prevent cross-word memory fetches that waste CPU execution cycles, the compiler automatically adds empty space bytes (**padding bits**) between misaligned parameters.

Ordering impact Example

```
struct abc
{
   char a;    ------>  1 Byte | 3 Bytes of internal padding inserted here
   int b;     ------>  4 Bytes
   char c;  ---->  1 Byte |  3 Bytes of trailing padding inserted here
} var;   ----> Total evaluated sizeof(var) = 12 Bytes!
```

[Image diagram showing alignment layout differences with padding slots across memory blocks]



Structure Packing

While padding enhances processing speed, it wastes memory space. To stop memory wastage, you can force the compiler to eliminate padding gaps using the #pragma pack(1) preprocessor directive.

#pragma pack(1) -----> Compresses structure to byte-level alignment

```
struct abc
{
   char a;  ------>  1 Byte
   int b;  ------>  4 Bytes
   char c; ------>  1 Byte
};  ------>  Total evaluated sizeof = 6 Bytes!
```

⚠ **Trade-off:** Structure packing saves physical RAM space, but it can degrade performance because the CPU may need multiple read cycles to access unaligned data members.

Note:

For making your understanding more clear and simple, you must visit the handwritten note provided in the same folder **13-Pointer**. It contains practical visual diagrams, annotations, and additional trace examples to solidify these structure and memory concepts.